

Git & GitHub Basics

Version control made simple: keep every version of your work, back it up in the cloud, and never again email a file named `final_v3_REALLY_FINAL.ipynb`.

1. The Big Idea

Git keeps a photo album of your project: every time you save a "snapshot" (called a commit), it remembers exactly what every file looked like at that moment. **GitHub** is a website where that album lives online, so your work is backed up and easy to share. **GitHub Desktop** is a free app that lets you do all of this with buttons and checkboxes; no typing commands required. It is all you need for this course.

Repository	A project folder that Git is watching ("repo" for short). The course materials live in one.
Clone	Download your own copy of a repo from GitHub to your computer.
Diff	A view of exactly what changed in a file: green lines were added, red lines were removed.
Commit	A saved snapshot of your changes, with a short message saying what you did.
Push / Pull	Push sends your new commits up to GitHub; pull brings down whatever is new on GitHub.



2. GitHub Desktop: All You Really Need

One-time setup

1. Create a free account at **github.com** (if you do not have one).
2. Download GitHub Desktop from **desktop.github.com** and install it.
3. Open it and sign in with your GitHub account when asked. That is it.

Cloning a repository (getting the course materials)

1. On the repo's page on **github.com**, click the green **Code** button, then **Open with GitHub Desktop**. (Or in the app: **File** → **Clone Repository** → **URL** tab and paste the repo's web address.)
2. Choose where on your computer the folder should go, and click **Clone**.
3. You now have the whole project on your machine. Open and edit the files with your normal tools (Jupyter, VS Code, Excel, anything).

Your everyday routine

1. **Start by pulling.** Open GitHub Desktop and click **Fetch origin** at the top. If it changes to **Pull origin**, click it again: this downloads anything new (for example, files your instructor added).
2. **Edit your files** as usual and save them.
3. **Review your changes.** Back in GitHub Desktop, the **Changes** list on the left shows every file you touched. Click a file to see its diff on the right: green lines are new, red lines were removed. This is the best way to double-check your own work.
4. **Choose what to save.** The checkbox next to each file decides whether it goes into this snapshot. Usually you leave them all checked.
5. **Commit.** In the box at the bottom left, type a short summary of what you did (for example, "Finish Day 5 exercises"), then click **Commit to main**.
6. **Push.** Click **Push origin** at the top to upload your commit to GitHub. Your work is now backed up.

Seeing your history: the **History** tab (top left) lists every commit ever made. Click any of them to see exactly what changed and when: your project's photo album.

Commit early, commit often. Small snapshots with clear messages ("Add sales chart", not "stuff") are easy to look back through, and every pushed commit means your work survives a lost or broken laptop.

3. Keeping Your Notes Safe When a Notebook Gets Updated

Suppose you add your own notes to a course notebook, and later the instructor fixes a typo in that same notebook and pushes the fix. Git now sees two different versions of one file, and merging notebooks is messy even for professionals. The fix is simple: never let it happen.

The golden rule: copy before you write

1. Before editing any course notebook, make a copy of it and add your name or `my-` to the filename: `Day05-1-Python-Review.ipynb` → `my-Day05-1-Python-Review.ipynb`.
2. Put all your notes and answers in the copy. Treat the originals as read-only.
3. Because the originals never change on your machine, **Pull origin** always works instantly: you get every instructor update, and your notes are untouched.

If you forgot and edited the original anyway

GitHub Desktop will complain when you try to pull. Do not try to merge; use this escape hatch instead:

1. **Save your work first.** In Finder / File Explorer, rename your edited notebook (add `my-` to the front), or copy it somewhere safe like your Desktop.
2. **Discard the original.** In GitHub Desktop, right-click the modified original in the **Changes** list and choose **Discard changes**. (Safe now, because your renamed copy holds all your work.)
3. **Pull.** Click **Pull origin**. You now have the instructor's fixed original plus your own copy with your notes, side by side.

Order matters: step 1 always comes first. As long as your copy is saved under a new name, nothing in steps 2 and 3 can lose your work.

4. The Same Moves at the Command Line

Everything GitHub Desktop does is a Git command underneath. You do not need these now, but you will see them in tutorials and documentation, so here is the translation:

<code>git clone https://github.com/user/repo.git</code>	Download a repo. (Desktop: File → Clone Repository)
<code>git status</code>	Which files have changed? (Desktop: the Changes list)
<code>git diff</code>	Show the changes line by line. (Desktop: click a file in Changes)
<code>git add notebook.ipynb</code>	Include this file in the next snapshot; <code>git add .</code> includes everything. (Desktop: the checkboxes)
<code>git commit -m "Finish exercises"</code>	Save the snapshot with a message. (Desktop: Commit to main)
<code>git push</code>	Upload your commits to GitHub. (Desktop: Push origin)
<code>git pull</code>	Download the latest from GitHub. (Desktop: Fetch origin / Pull origin)
<code>git log --oneline</code>	List past commits, one per line. (Desktop: the History tab)

MSBA Bootcamp · Day 5 · Git & GitHub Basics — companion sheet to Command Line Basics; the routine to remember is **pull** → **edit** → **review the diff** → **commit** → **push**.